UNIVERSITÄT BAYREUTH

Mesh-to-CAD AI:A Continuation

Modelling Seminar

at

University of Bayreuth

Chair of
Scientific Computing
in cooperation with

Chair of Engineering Design and CAD

Written by
Harshitha Sai Nallagonda, Nisarg Pawar
Supervised by
Prof. Dr.-Ing. Tobias Rosnitschek
26 October 2026

Contents

1 Introduction **1**
1.1 Problem statement 1
1.2 Aim 1

2 Existing Literature **3**
2.1 Mesh To CAD AI 3
2.2 CAD Coder 4
2.3 TEXT2CAD 5

3 Methodology **6**
3.1 Data Generation 7
3.2 STL-Derived Features for LLM 11
3.3 LLM-Based Learning 12
3.3.1 Baseline Model 12
3.3.2 Fine-tuning with single-view 12
3.3.3 Fine-tuning with geodata 13
3.4 Shape optimisation 14
3.5 Comparison of the Mesh-to-CAD AI (2025) and Mesh-to-CAD AI:A Continuation (2026) Methodologies 24

4 Results and Discussion **26**
4.1 Baseline Model 26
4.2 Fine-tuning with single-view 27
4.3 Fine-tuning with geodata 29
4.4 Shape optimisation results 30
4.5 Evaluation on Unseen Data 34

5 Conclusion and Future Work **37**

References **38**

Chapter 1

Introduction

Topology optimisation helps us optimise the layout of the material in a specific design space for a given set of conditions. Currently used at the concept level of the design process, Topology Optimisation produces meshed STL parts. An STL file is a mesh of triangles and contains no information about the design parameters or design history. This becomes a problem later during fine-tuning and the manufacturing process, as they can not be used in the CAD environment. The Existing Geometry Converter gives us highly fragmented or randomly segmented facets which are hardly usable. A parametric CAD code contains design history with editable dimensions and operations. This can be easily edited during the design and manufacturing process. Hence, there is a need to convert the STL file into the editable CAD code.

1.1 Problem statement

During the design and manufacturing process, we have the STL file that is obtained from topology optimisation. An STL file, being non-editable and with no design history, needs to be converted into editable CAD code to be used in the CAD environment. The Parametric CAD code is also helpful during the fine-tuning and shape optimisation process.

1.2 Aim

The idea is to make use of a Large Language Model (LLM) that can reconstruct the parametric CAD code from the given STL file and later improve

the results using shape optimisation. Thus, the goal of this project is to use AI tools, specifically an LLM, to perform the STL-to-CAD conversion to obtain editable CAD code and to improve the results using shape optimisation.

Parametric CAD code Example:

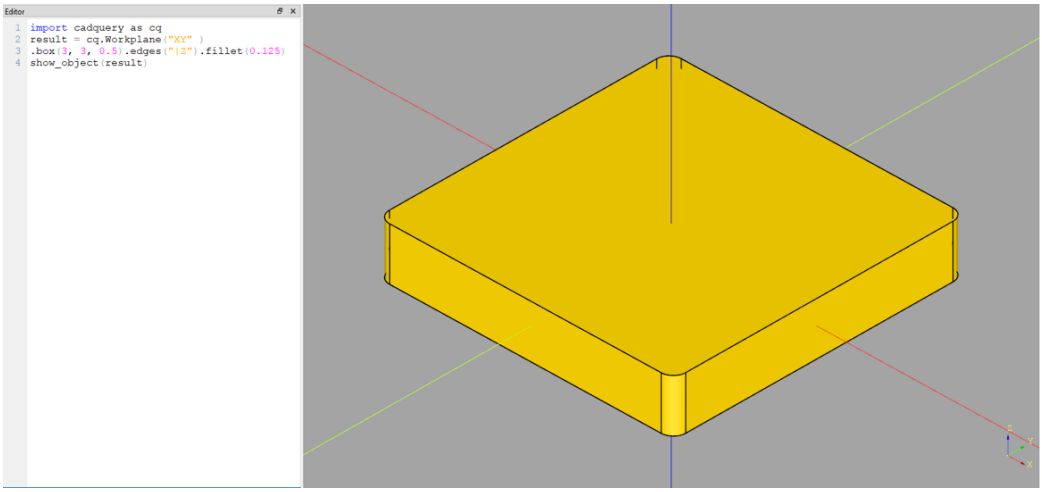


Figure 1.1: CadQuery editor

Chapter 2

Existing Literature

In the following section, various literature sources that have used different methods to obtain a parametric CAD code from the STL file will be discussed. This literature review is helpful and gives an overview of the techniques that were adapted in the methodology of this project.

2.1 Mesh To CAD AI

This approach uses Vision Language Models (VLMs) to generate parametric, editable CAD models from the non-editable STL meshes input. Firstly, the 3D geometric data was represented as a 3x3 matrix of 2D images. The images are computationally less expensive than directly using the STL file, making them a better choice as input. So, the images are different views of the object in the STL file. The images are then interpreted by a finely tuned Qwen 2.5-VL model, which produces a non-structured text description. Later, the generated unstructured text is converted into a structured CAD sequence using Text2CAD. This obtained CAD sequence is transformed into a Python script to be finally executed in FreeCAD. The approach also incorporates an optimisation algorithm into the Python code to verify that the final CAD model matches the input model. Thus, the approach combines Qwen 2.5 VLM and Text2CAD to obtain the output. The approach worked for simple geometries, and shape optimisation led to good results. However, the system struggles to generalise complex, topology-optimised parts. The system also struggles with parts that are not included in its training data [8].

2.2 CAD Coder

CAD Coder is an open-source vision-language model that automatically generates parametric CAD code from sketches, drawings, and engineering images. It uses a multimodal architecture combining a vision encoder to extract geometric features and a language model decoder to generate CAD scripts. The system also uses parameter-efficient fine-tuning (PEFT) with Low-Rank Adaptation (LoRA). The training dataset consists of paired CAD sketches and their related parametric modelling scripts. The training dataset is a 163k CAD-model image and code pairs dataset. Each CAD command is represented by

$$c_i = (t_i, p_i),$$

where

$$c_i \in \mathbb{R}^{17}, \qquad t_i \text{ is the command type, and } p_i \in \mathbb{R}^{16}.$$

The architecture of the CAD Coder model is shown in Figure 2.1.

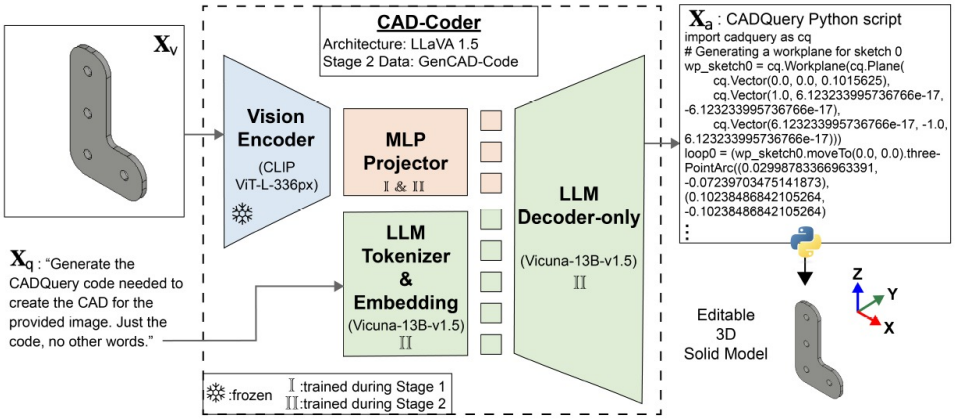


Figure 2.1: CAD Coder Architecture [5]

It has a 100% valid syntax rate, which means that the generated Python script is syntactically correct and will not contain any syntax errors. It uses the LLaVa1.5 Architecture with the Vicuna-13B-v1.5 LLM and CLIP-ViT-L-336px Vision encoder. The Vision encoder takes images of 336pixels × 336 pixels size to capture finer details, and the Vicuna LLM has 13 Billion Parameters. There is a two-layer Multi-Layer Perceptron that connects the Vision encoder and the LLM. The two-layer MLP is trained to maximise the autoregressive likelihood function:

$$p_\theta(X_a \mid X_v, X_q, X_a) = \prod_{i=1}^{L} p_\theta\left(x_i \mid X_v, X_q, X_{a,<i}\right)$$

where X_a: answer/caption, X_v: image, X_q: question, x_i: i-th token of the answer, L: answer length, θ: model parameters, $X_{a,<i}$: answer tokens before position i, p_θ: model predicted probability.

Because of the LLaVa architecture, there are no intermediate conversions or steps necessary, as the input image is directly converted to a parametric CAD script. Hence, because of this and its architecture, the CAD Coder model will be used as the base model for this project [5].

2.3 TEXT2CAD

This approach directly creates CAD models from the given natural language instructions. Text2CAD formulates CAD generation as a sequential task, where models predict step-by-step CAD operations. The system converts language instructions to structured CAD actions like sketching, cutting, and applying geometric constraints. It was trained on paired datasets with text descriptions and corresponding CAD operation sequences. The dataset contains approximately 170K models and 660K text annotations. Because of the availability of such a large dataset with varying geometric complexities, it was used as a reference for dataset creation in this project [7].

Summary: Together, this literature provides the foundation of the project. First, Mesh to CAD AI was used as a reference for creating views of the STL files and for the shape optimisation process. The CAD Coder model was used as the pretrained VLM model, while Text2CAD was referred to during the dataset creation step of this project. Hence, to summarise this section, the above literature was used in this project in different ways, as discussed above.

Chapter 3

Methodology

The goal is to make use of an LLM to obtain a parametric CAD code from the given STL file. An editable CAD code is needed to change the parameters during the design and manufacturing process. So, the methodology of this project is divided into the following steps:

Step 1: Data Generation

The dataset creation is the first step of this project. The purpose of this step is to have objects with varying geometric complexities for later analysis. So, firstly, design a set of 20 heterogeneous parts consisting of various CAD operations in FreeCAD. Then, using the Python interface, approximately 100 variants of each part are generated by varying geometric parameters. Thus, for each variant, we have a CAD file, a FreeCAD Python script, and an STL mesh, and then data preprocessing is performed.

Step 2: LLM-Based Learning

In this step, the LLM is trained to learn the corresponding mapping from the input to the CAD script, and the model is fine-tuned using techniques such as Low-Rank Adaptation (LoRA). Then, the CAD scripts that are generated will be executed in FreeCAD to obtain CAD models.

Step 3: Shape optimisation

After the CAD scripts are generated, the geometric accuracy of the generated CAD models is evaluated, where parameters like dimensions and features are adjusted and optimised. The performance of the framework is later evaluated using geometric accuracy and robustness.

Step 4: Analysis

The final step is to perform analysis by investigating factors like the number of CAD operations, performance of the LLM, effects of fine-tuning parameters, effects of shape optimisation on final accuracy, and so on. This will be discussed in Section 4, i.e., Results and Discussion.

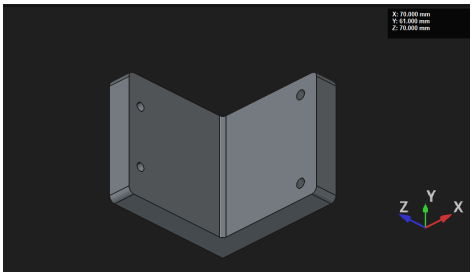
3.1 Data Generation

Initially, 20 heterogeneous parts were created in FreeCAD, which had varying complexities and between 2 and 8 CAD Operations. Then 100 variants of each of these parts were generated with the FreeCAD Python Interface, where the parameters were changed. Each variant has a CAD file, a parametric CAD, and an STL mesh. The purpose of this step is to analyse the data or performance with respect to complexity and the number of CAD operations. These 20 parts with 100 variants create a dataset of 2000 objects. Then, 9 views for each of the above 2000 objects were generated to use the image as the input to the LLM. The image is given as the input to the LLM, as it is computationally cheaper compared to giving an STL file. The objects are categorised into various complexity levels for a better understanding and analysis of the LLM's performance, as shown in the following table:

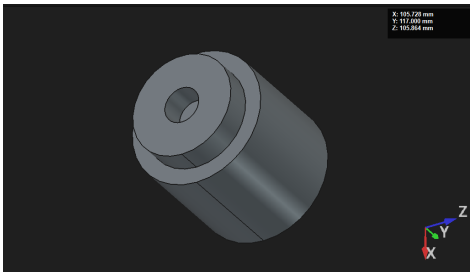
Level	**Index**	**Sketch Complexity**	**Geometry Complexity**
Simple	1	1–6 entities; basic lines, circles, rectangles	1–3 features; basic blocks, simple holes
Medium	2	7–15 entities; several lines/arcs	4–7 features; holes, fillets, curved surfaces
High	3	16+ entities; many arcs, complex profiles	8+ features; many surfaces, complex curves

Table 3.1: Classification of Sketch and Geometry Complexity

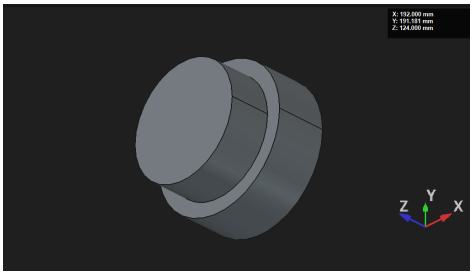
The 20 heterogeneous parts that were created are shown below:



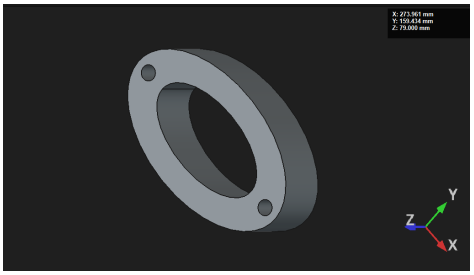
(a) Part 1



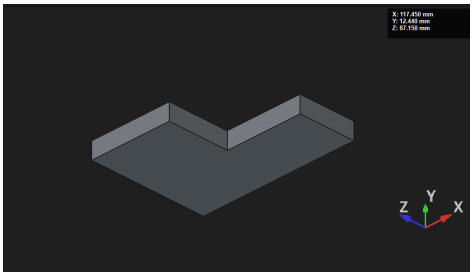
(b) Part 2



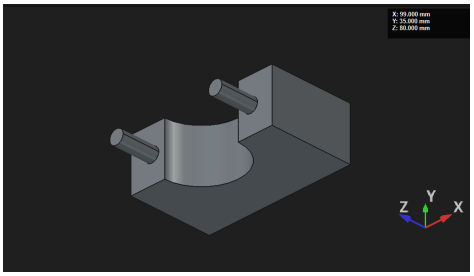
(c) Part 3



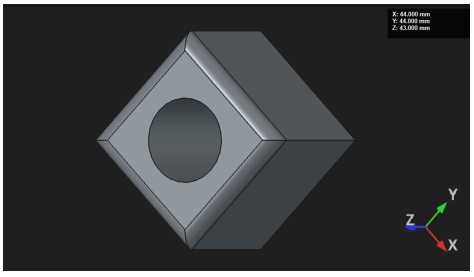
(d) Part 4



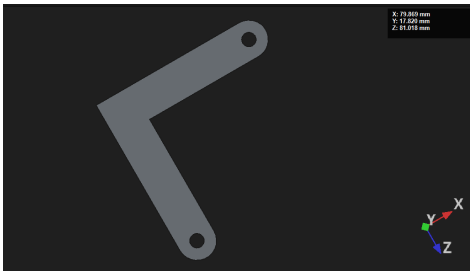
(e) Part 5



(f) Part 6



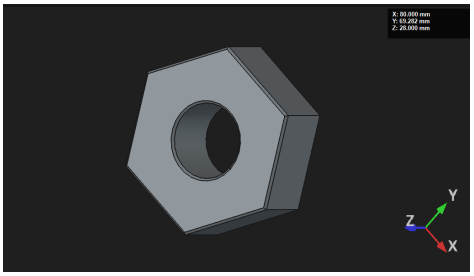
(g) Part 7



(h) Part 8

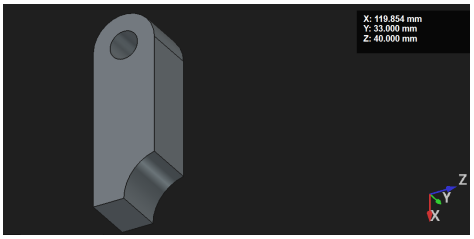


(i) Part 9

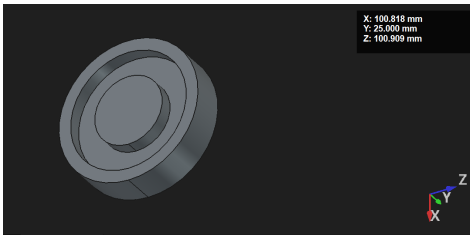


(j) Part 10

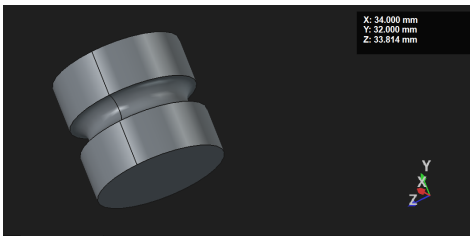
Figure 3.1: Heterogeneous Parts 1–10



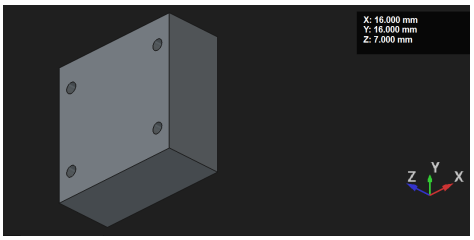
(a) Part 11



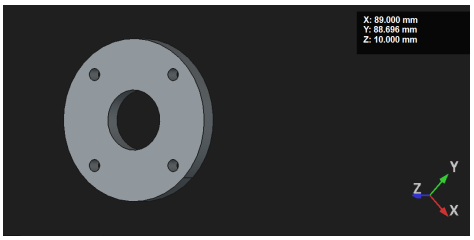
(b) Part 12



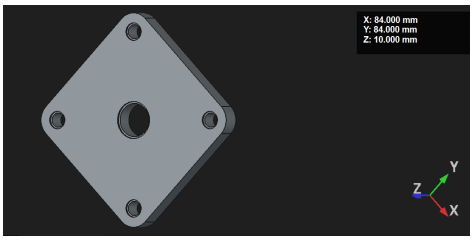
(c) Part 13



(d) Part 14



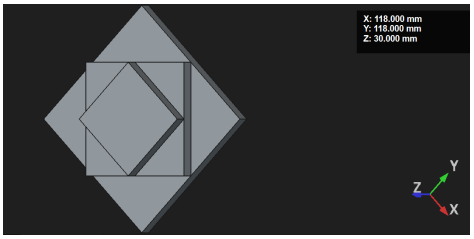
(e) Part 15



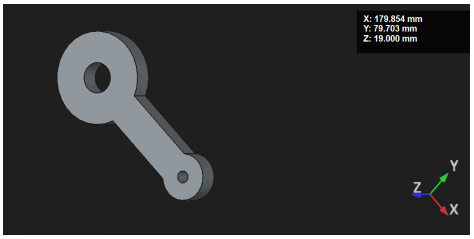
(f) Part 16



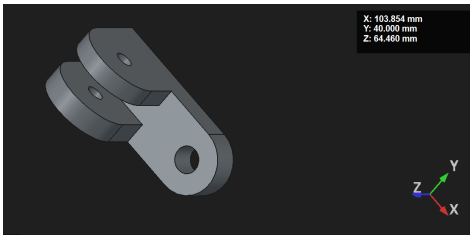
(g) Part 17



(h) Part 18



(i) Part 19



(j) Part 20

Figure 3.2: Heterogeneous Parts 11–20

Part	CAD Operations	Sketch Complexity	Geometry Complexity
1	8	Medium	High
2	4	Medium	Medium
3	4	Medium	Medium
4	4	Medium	High
5	2	Simple	Simple
6	4	High	High
7	5	Simple	Medium
8	2	Medium	High
9	4	Medium	High
10	6	Medium	Medium
11	2	High	High
12	4	Medium	High
13	4	Medium	High
14	4	Simple	Simple
15	2	Medium	Medium
16	4	Medium	High
17	2	High	High
18	6	Simple	High
19	2	High	High
20	7	High	High

Table 3.2: CAD model complexity and CAD operation counts

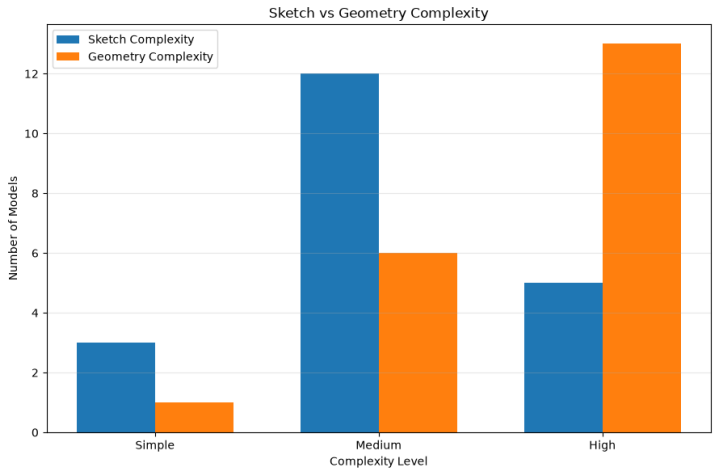


Figure 3.3: Data Distribution with respect to complexity

The distribution of the 20 heterogeneous parts with respect to different sketch

and geometric complexity is shown in Figure 3.3. It can be observed that there are more objects of medium to high complexity. As existing models perform well with simple objects, the goal was to make the LLM learn more from medium and complex objects rather than having more simple objects in the dataset.

3.2 STL-Derived Features for LLM

Approaches and Challenges:

The first approach was to investigate whether the parametric CAD models could be constructed from the geometric information found in STL meshes. STL files contain objects as triangulated surfaces and do not have any information about original sketches, design parameters, or feature history. So, various methods were explored to extract useful geometric information for large language models (LLMs).

Firstly, using Trimesh, the geometric properties were extracted. It helps in computing properties like volume, bounding box sizes and surface area. The extracted data, along with the STL file, have been given to current large language models (LLMs), including Gemini and Chatbot, to generate CadQuery scripts. These were then used to generate STEP models. CadQuery is a Python library for building parametric 3D CAD models. With a simple-to-use interface, this method gives good results for simple objects, preserving the geometric properties. However, it is not satisfactory for complex objects.

Next, a Python script was created to extract mesh data such as bounding-box dimensions, vertex and triangle counts, and sample vertex coordinates using the Three.js JSON export method. However, this information was insufficient to reconstruct complex part designs. Other advanced techniques using Trimesh, Open3D, and RANSAC were explored to identify geometric shapes like planes and cylindrical surfaces. These methods provided geometric information, but the conversion into proper sketches, constraints, and extrusion operations was challenging.

These experiments showed that extracting the geometric data from STL files is possible, but not sufficient for converting this information into accurate parametric CAD models. Thus, another approach was selected in which the images of CAD parts are provided as input to a pre-trained LLM.

3.3 LLM-Based Learning

3.3.1 Baseline Model

As discussed in the Literature section, the CAD Coder model is used in this project. So, after installing the CAD Coder model, a few sample images were given to the model to obtain a baseline model estimate. Various single-views of the object were tried, along with a combined-views image as input. When the combined-views image was given as an input to the model, the model did not perform better.

Reason: This is because CAD Coder uses the LLaVA architecture, and it does not support multi-view inputs. LLaVA architecture works only with single-view images. So, we later tried to perform analysis of the baseline model with single-view image input, and the results were satisfactory that will discussed in section 4.1. The model could reconstruct the object but did not recreate the object entirely.

Hence, this baseline model analysis helped us to understand that the input given should be a single-view image and not a combined-view, and that it does work well when new unseen data is given.

Next, we will discuss the various methods of fine-tuning with different parameters that were implemented. The LLM is fine-tuned with various parameters and fitted to our dataset.

3.3.2 Fine-tuning with single-view

The CAD Coder model was fine-tuned with a single-view image as the input. The fine-tuning is done using low-rank Adaptation (LoRA). It helps in fine-tuning the LLM to perform well on targeted tasks. An additional layer of trainable parameters is introduced while keeping the original parameters frozen, and the new parameters are of a substantially reduced rank. Finally, the results of the original network and the low-rank network are computed with a dot product and then used to compute the loss. The entire dataset is split into a 70:30 ratio for train and test split. So, 70 variants out of 100 for each variant were used for training, and the remaining were used for testing. The model was trained on both images and CAD code, and testing was done on the images. The training took 1 hour on the university's HPC FESTUS server with one L40 GPU node. The fine-tuning was performed using Low-Rank Adaptation (LoRA) with the following parameters:

- Rank: 128

- LoRA alpha: 256
- LoRA dropout: 0.05
- Learning rate: 1×10^{-5}
- Epochs: 3

The output obtained after testing is CadQuery, which is a parametric CAD code and the results will be discussed in section 4.2. The model worked fine with the data, but other parameters and various training techniques were also tried, which will be discussed next.

3.3.3 Fine-tuning with geodata

The CAD Coder model was fine-tuned with a single-view image as the input. Geodata of the object was also incorporated in this fine-tuning. Geodata, or Geometric Data, contains information about the object such as volume, surface area, overall dimensions, and so on. The entire dataset is split into a 70:30 ratio for train and test split. So, 70 variants out of 100 for each variant were used for training, and the remaining were used for testing. The model was trained on the image, its geodata, and CAD code, and the testing was on the image mapped with its geodata. The training took 2 hours on the university's HPC FESTUS server with one L40 GPU. The fine-tuning was performed using Low-Rank Adaptation (LoRA) with the following parameters:

- Rank = 128
- LoRA alpha = 256
- LoRA dropout = 0.05
- Learning rate = 1×10^{-5}
- Epochs = 3

The output obtained after testing is CadQuery, which is a parametric CAD code and the results will be discussed in section 4.3.

Another fine-tuning was also performed by incorporating the scales and axes in the images, but it did not produce satisfactory results.

In the fine-tuning with geodata incorporated, the model seems to perform better and reconstructs the object in an optimal number of CAD operations. Hence, this fine-tuned model is chosen for the next step of shape optimisation.

3.4 Shape optimisation

The aim of shape optimisation is to improve the geometric similarity between a CAD model generated by CAD Coder and a reference STL model. The CAD model is parameterised, and its geometric parameters are adjusted iteratively till the difference between them is minimised.

Geometric Similarity Metrics for CAD Reconstruction

The Hausdorff and Chamfer distances are used to measure the geometric similarity between the reconstructed CAD model and the reference STL. The Hausdorff distance identifies the maximum deviations between the surfaces, while the Chamfer distance measures the average deviations.

Hausdorff Distance

The Hausdorff distance measures the maximum distance between points on two geometric surfaces. For point sets A and B, the Hausdorff distance is defined as

$$H(A, B) = \max\{h(A, B), h(B, A)\},$$

where

$$h(A, B) = \max_{a \in A} \min_{b \in B} \|a - b\|.$$

A smaller Hausdorff distance shows less deviations, whereas a large value implies more local deviations.

Chamfer Distance

Chamfer distance measures the average nearest-neighbour distance between two point sets:

$$CD(A, B) = \frac{1}{|A|} \sum_{a \in A} \min_{b \in B} \|a - b\|^2 + \frac{1}{|B|} \sum_{b \in B} \min_{a \in A} \|b - a\|^2.$$

Unlike Hausdorff distance, Chamfer distance considers the average deviation across the surfaces, making it less sensitive to outliers.

Steps to calculate Hausdorff and Chamfer distances

We use `trimesh` to sample points from the mesh surfaces and `scipy.spatial.cKDTree` to find nearest-neighbours efficiently.

1. Orientation and centring

   Align the STL and STEP models to the same orientation and centre their origins.

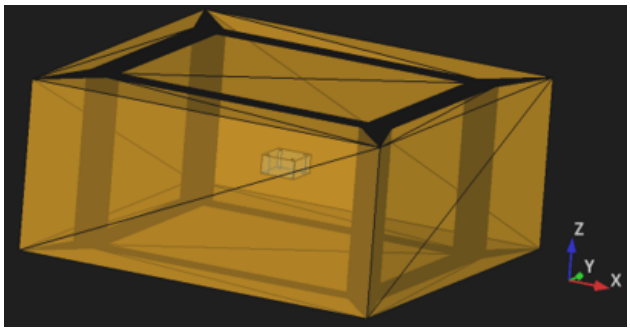


Figure 3.4: STEP (Blue / Small model) & STL (Brown / Big model) orientation

2. Sample the surfaces

   We sample 10,000 points from the STL surface and 10,000 points from the STEP surface.

   These points are randomly selected surface locations, not fixed corresponding points. The sampled points on both surfaces are shown in Figure 3.5.

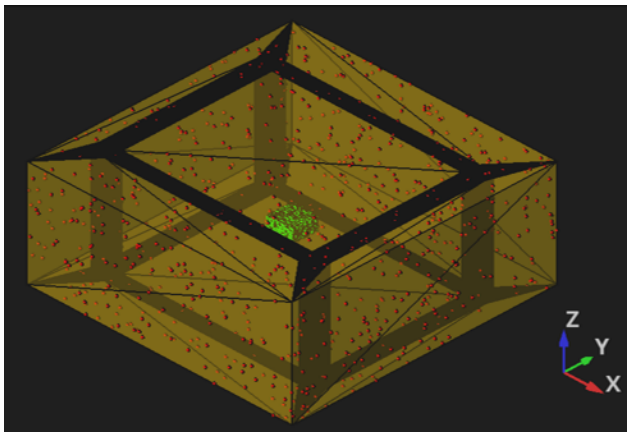


Figure 3.5: Sampled points on STEP (green points) & STL (red points)

3. Find nearest-neighbours

For every sampled STL point, we find its nearest sampled STEP point. Figure 3.6 shows the distance lines between the nearest-neighbour points.

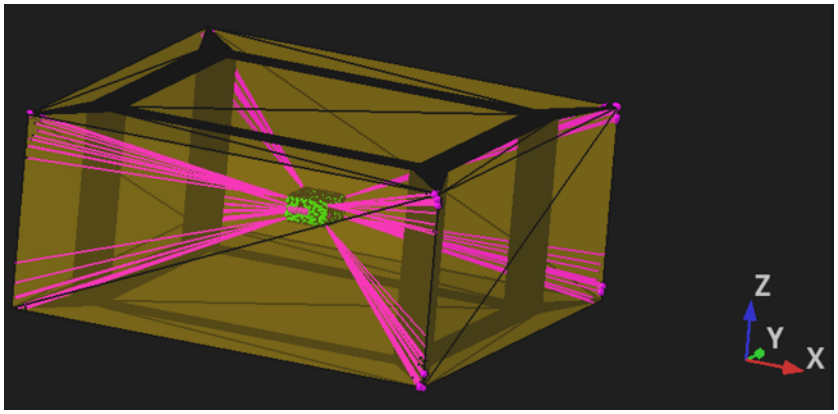


Figure 3.6: Distance lines before shape optimisation

4. Distance calculation

   Calculate the Euclidean distances between corresponding nearest points:

$$d(\mathbf{p}, \mathbf{q}) = \|\mathbf{p} - \mathbf{q}\|_2 = \sqrt{(p_x - q_x)^2 + (p_y - q_y)^2 + (p_z - q_z)^2}.$$

5. Metric evaluation

   Calculate Hausdorff distance (maximum deviation) and Chamfer distance (average deviation).

Geometric Alignment and Baseline Evaluation

Before performing shape optimisation, the geometric orientation of the CAD Coder-generated model must be aligned with the reference STL model. Since CAD Coder may generate models with different orientations and significantly different scales, direct comparison of the two geometries may produce large distance errors that show both scale and shape differences.

To solve this issue, an automatic evaluation process is written in Python using FreeCAD. First, the reference STL and the generated STEP model are loaded, and their bounding-box dimensions are calculated. A uniform scaling factor is then determined from the ratio of their maximum bounding-box dimensions. This factor is applied to a temporary copy of the STEP model, so that rotations can be evaluated using the normalised geometry without modifying the original model.

The scaling factor is calculated as

$$s = \frac{\max(L_{\mathrm{STL}}, W_{\mathrm{STL}}, H_{\mathrm{STL}})}{\max(L_{\mathrm{STEP}}, W_{\mathrm{STEP}}, H_{\mathrm{STEP}})},$$

where L, W, and H denote the bounding-box dimensions along the three coordinate axes.

The rotation is selected by testing different combinations of 0°, 90°, 180°, and 270° about the three coordinate axes. For each candidate, the relative bounding-box error with respect to the reference STL is calculated. The rotation of the scaled STEP model giving the smallest error is selected as the best candidate. This approach helps in the orientation-selection process based on the bounding-box dimensions.

The relative bounding-box error is defined as

$$E_{\mathrm{BB}} = \sum_{i \in \{x,y,z\}} \frac{\left| D_i^{\mathrm{STEP}} - D_i^{\mathrm{STL}} \right|}{|D_i^{\mathrm{STL}}|}$$

where D_i represents the bounding-box dimension along each axis.

Once the optimal rotation combination has been obtained, it is then applied to the original, unscaled STEP model. The bounding-box centres of the rotated STEP model and the reference STL are then aligned through translation. No scaling is applied to the geometry used for the baseline evaluation, thus preserving the original dimensions generated by CAD Coder.

The aligned, original-scale STEP model is then compared with the reference STL by calculating the Chamfer and Hausdorff distances. These metrics quantify the average surface deviation and the maximum sampled nearest-neighbour deviation, respectively. The resulting values give the baseline geometric error before performing shape optimisation.

Visual Verification of the Selected Orientation

Along with this automatic evaluation, a separate FreeCAD macro was created to visually check the selected orientation. The macro applies the selected rotation to the original STEP model, aligns its bounding-box centre with the reference STL by translating the STEP to the STL centre. The resulting orientation is shown in Figure 3.7, Figure 3.8 and Figure 3.9.

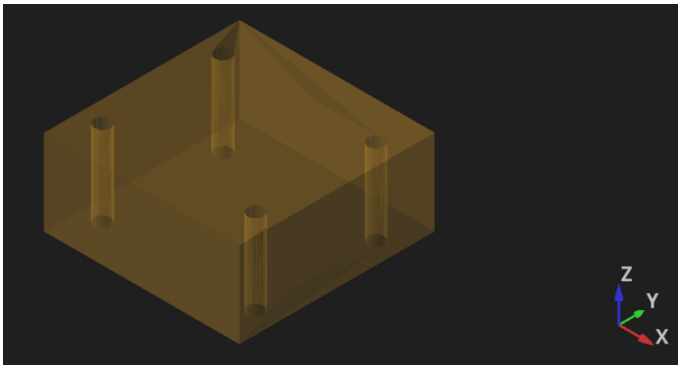


Figure 3.7: Reference STL

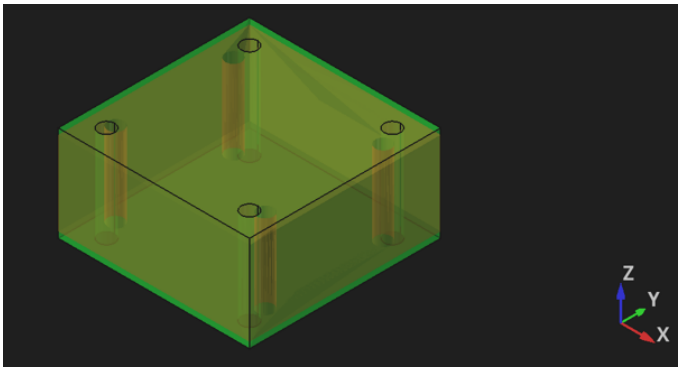


Figure 3.8: Scaled STEP (green model) & Reference STL (brown model)

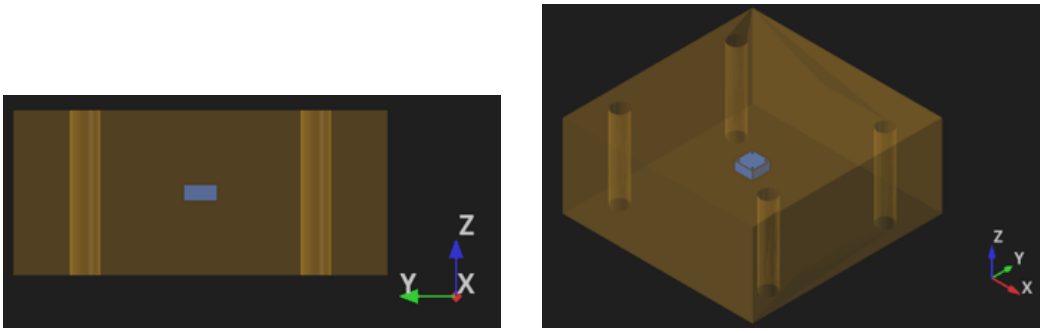


Figure 3.9: Front (left) and isometric (right) views of the unscaled STEP (blue model) and reference STL (brown model).

Shape Optimisation Procedure

Objective

The model generated by CAD Coder may capture the geometry of the target STL model, but its dimensions may not match. To solve this issue, shape optimisation is used to find the best parameter values that minimise geometric differences.

Shape Optimisation Pipeline

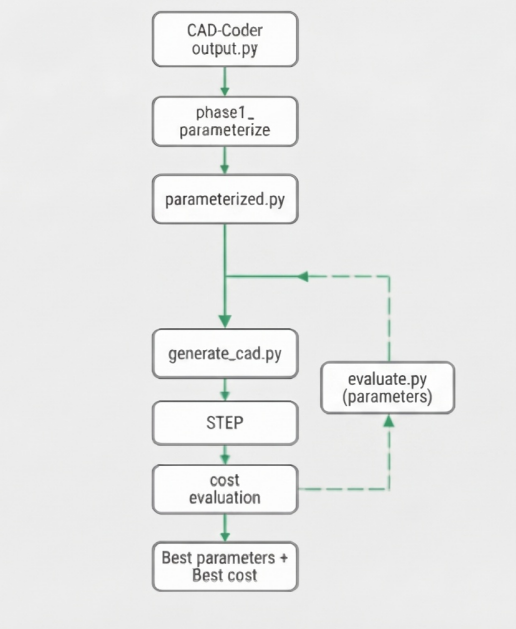


Figure 3.10: Shape Optimisation Pipeline

1. Parameterisation of the CAD Model

   The output generated by CAD Coder is a CadQuery script, which is then processed to identify numerical values of geometric operations. `phase1_parameterize.py` identifies numerical geometry parameters and replaces suitable values with named parameter variables. The resulting output file, `parameterized.py`, can generate different versions of the same geometry.

2. CAD Generation

   In this step, the script generates the STEP files from the parameterised CadQuery Python script. It receives parameter values from the optimisation algorithm, executes the CadQuery code in an external Python environment, and exports the resulting geometry as a STEP file.

3. CAD–STL Cost Evaluation

Before calculating the cost, the STEP model and STL model must be placed in a comparable coordinate system, similar to the process in baseline evaluation. The unmatched volumes are calculated to obtain the cost.

4. Optimisation Feedback and Final Result

   The calculated cost is returned to `evaluate.py`. The evolutionary optimisation algorithm uses the cost to select better candidates and generate subsequent generations. In the end, the best parameter vector is found and its cost is returned.

The optimisation problem can be expressed as

$$\mathbf{x}^* = \arg\min_{\mathbf{x}} C(\mathbf{x})$$

where:

- $\mathbf{x}$ is the vector of CAD parameters.
- $C(\mathbf{x})$ is the cost that measures the difference between the generated CAD model and the target STL.
- $\mathbf{x}^*$ is the parameter vector that produces the lowest cost found by the evolutionary optimisation algorithm.

Cost Function

The quality of each candidate CAD model is evaluated using the volume that does not overlap between the CAD model and the target STL model.

Let V_{CAD} denote the CAD solid and V_{STL} denote the target solid. The unmatched volumes are defined as

$$V_{\text{CAD only}} = V_{\text{CAD}} \setminus V_{\text{STL}}$$

$$V_{\text{STL only}} = V_{\text{STL}} \setminus V_{\text{CAD}}$$

Here, CAD-only represents the volume present in the CAD model but absent from the STL, while STL-only represents the volume present in the STL but absent from the CAD model.

The cost function implemented in our pipeline is

$$C(\mathbf{x}) = \frac{V_{\text{CAD only}} + V_{\text{STL only}}}{V_{\text{STL}}}$$

Normalising by the STL volume makes the cost dimensionless and expresses the total unmatched volume relative to the target volume.

A zero cost means a perfect volume match, while larger values indicate larger differences. Therefore, the optimisation algorithm aims to minimise the cost function.

Evolutionary Optimisation Algorithm

An evolutionary algorithm is used because the relationship between the CAD parameters and the resulting geometric cost is nonlinear. Variations in dimensions may lead to invalid geometries.

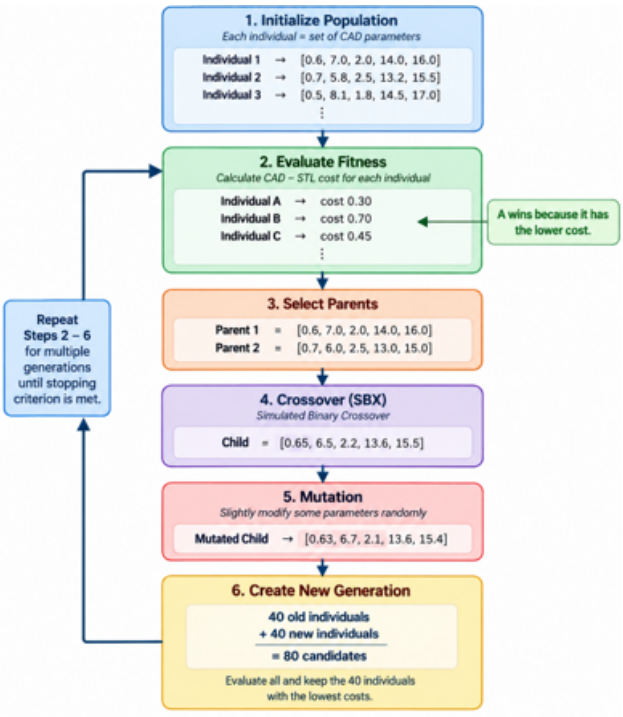


Figure 3.11: Evolutionary Algorithm Steps

The algorithm searches for improved parameter combinations through the following steps:

1. Initialize the population

Generate multiple candidate parameter vectors within their allowed bounds. The initial extracted CAD parameters can be included as the starting candidate.

2. Evaluate the population

   Generate a CAD model for each candidate, align it with the target STL, and calculate its cost. Lower-cost candidates represent better geometric matches.

3. Tournament selection

   Randomly select a small group of candidates and choose the one with the lowest cost as a parent. Repeat to select parents for reproduction.

4. Simulated Binary Crossover (SBX)

   Combine two parent parameter vectors to generate offspring that inherit parameter values influenced by both parents.

5. Polynomial mutation

   Randomly modify selected parameter values to introduce variation and explore new regions of the search space.

6. Survivor selection

   Combine the parent population and offspring, then retain the candidates with the lowest costs to form the next generation.

7. Termination and final result

   Repeat the evaluation and reproduction process for the specified number of generations. Return the best parameter vector and its corresponding cost.

Parameter Bounds

To prevent unrestricted changes to the original geometry, each parameter is optimised within predefined lower and upper bounds.

The 2025 project [8] used the following ranges based on experimental observations:

Parameter type	**Variation**
Sketch dimensions	±20%
Extrusion dimensions	±95%

The 2025 project report [8] explains that large variations in sketch dimensions can distort the original sketch, whereas extrusion lengths can tolerate larger variations without necessarily destroying the geometry.

Visual results after shape optimisation

The results after shape optimisation are visually shown in Figure 3.12, Figure 3.13 and Figure 3.14. As shown in the Figure 3.14, the distance lines are significantly reduced in comparison to Figure 3.6, indicating that the process effectively minimised deviation.

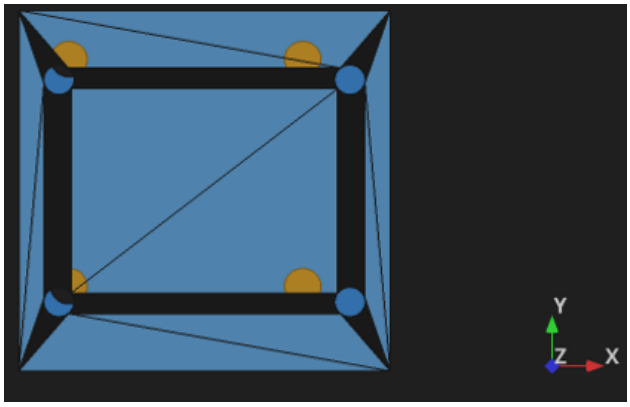


Figure 3.12: STEP (Blue model) & STL (Brown model) orientation

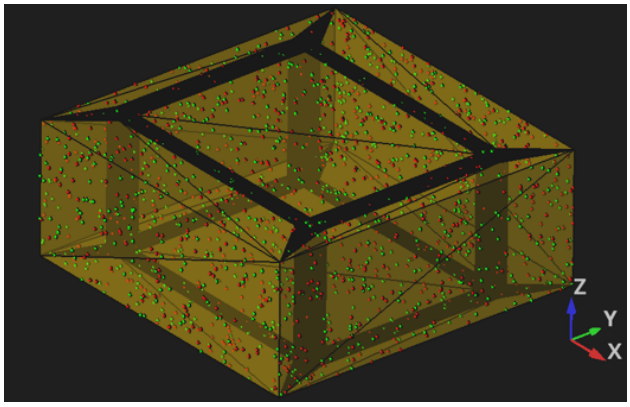


Figure 3.13: Sampled points on STEP (green points) & STL (red points)

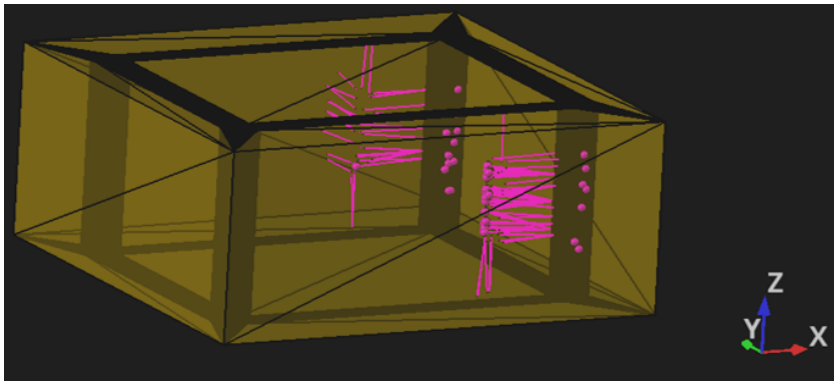


Figure 3.14: Distance lines after shape optimisation

3.5 Comparison of the Mesh-to-CAD AI (2025) and Mesh-to-CAD AI:A Continuation (2026) Methodologies

The main objective of both projects is to obtain a parametric CAD code from an input STL model. The differences lie in their processing pipelines and implementation is shown in Table 3.3.

Comparing these two approaches shows the different techniques used to achieve the same goal: generating parametric CAD models that represent the geometry of an input STL mesh. The 2025 approach relies on intermediate steps and conversions, whereas the 2026 approach focuses to simplify the process through direct image-to-code generation.

Aspect	**Mesh-to-CAD AI (2025)**	**CAD Coder (2026)**
Main Goal	Generate parametric CAD code from an input STL model.	Generate parametric CAD code from an input STL model.
Input representation	STL mesh rendered into multiple views.	STL mesh rendered into single views, along with extracted geometric metadata from STL.
Input image processing	Fine-tuned Qwen 2.5 VLM generates a textual description from the rendered views.	CAD Coder's vision encoder processes the rendered images to provide visual features for code generation.
Intermediate representation	The Text2CAD approach converts the textual description into a CAD sequence.	No intermediate text description or CAD sequence is required.
CAD code generation	A custom conversion algorithm converts the CAD sequence into an executable Python FreeCAD script.	Fine-tuned CAD Coder generates CadQuery scripts directly from the input images.
Shape optimisation	After initial CAD model generation, the model is optimised against the input STL by adjusting sketch dimensions and extrusion parameters to minimise geometric mismatch.	The evolutionary optimisation concept is kept, but changed the CAD representation, parameterisation, generation pipeline and some aspects of the cost evaluation.
Final output	Parametric CAD model generated in FreeCAD.	Parametric CAD model generated by CadQuery.

Table 3.3: Comparison of the Mesh-to-CAD AI (2025) and Mesh-to-CAD AI:A Continuation (2026) Methodologies

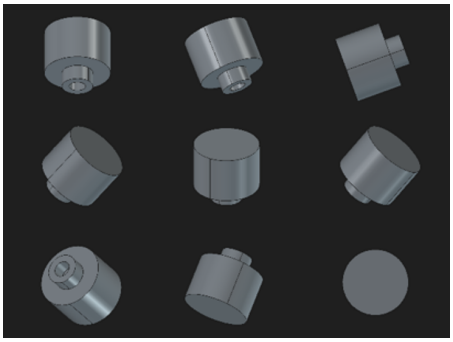
Chapter 4

Results and Discussion

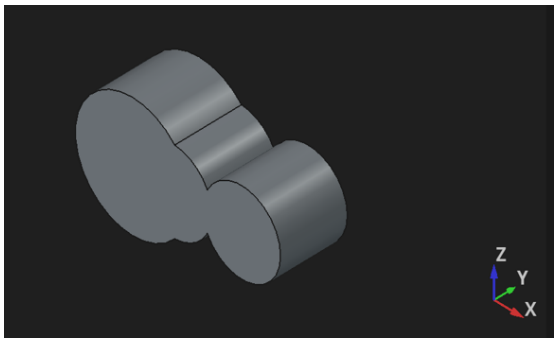
The results obtained from the model with different fine-tuning parameters will be discussed in this section.

4.1 Baseline Model

Because of the LLaVA architecture, when the combined-views image was given as input to the model, as discussed in section 3.3.1, the model could not perform better, as shown below.

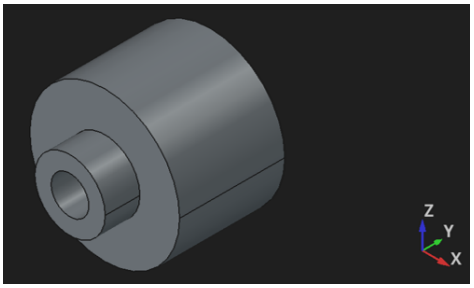


(a) Part 2 Input with Volume: 1996293.500000 mm^3

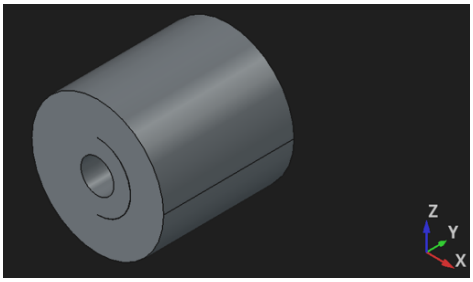


(b) Part 2 Output STEP with Volume: 0.062432 mm^3

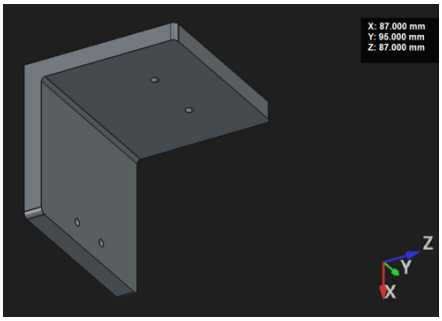
The baseline model with single-view image input gave satisfactory results, as shown below. However, there is a huge difference in the volume between the original and generated files. This can be observed in the following examples.



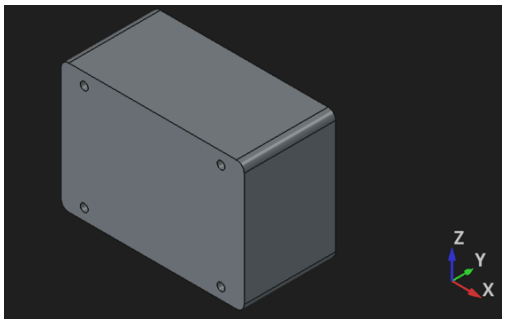
(a) Part 2 Input with Volume: 1996293.500000 mm^3



(b) Part 2 Output STEP with Volume: 0.316644 mm^3



(a) Part 1 Input with Volume: 155236.578125 mm^3

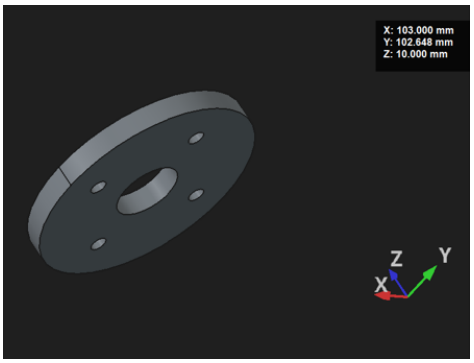


(b) Part 1 Output STEP with Volume: 1.178165 mm^3

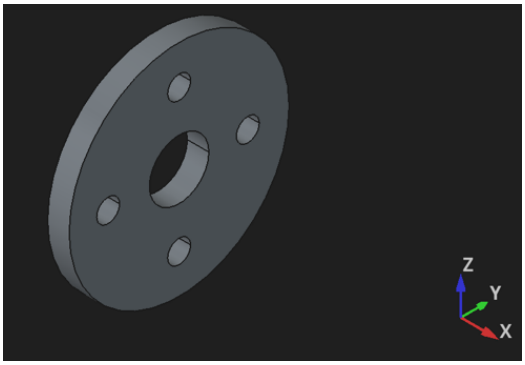
Hence, this analysis helped in understanding the need to give the input as a single-view image and not a combined-view.

4.2 Fine-tuning with single-view

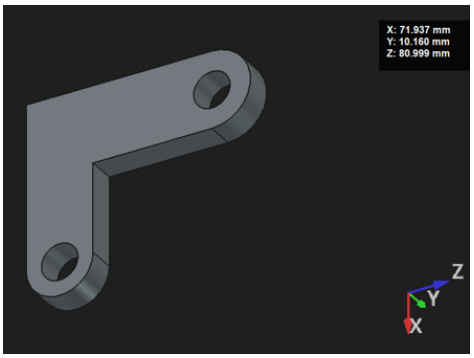
After performing the fine-tuning process as discussed in section 3.3.2, the obtained results are shown and discussed next.



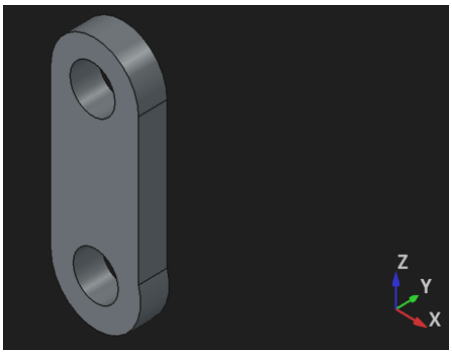
(a) Part 15 Input with 2 CAD ops, Volume: 155236.578125 mm³



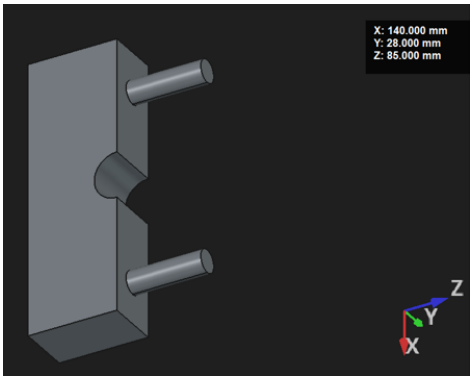
(b) Part 15 Output STEP with 2 CAD ops, Volume: 0.236588 mm³



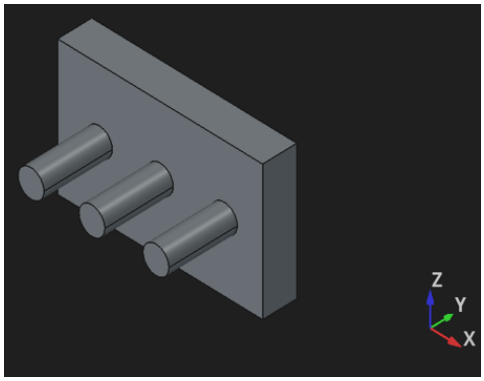
(a) Part 8 Input with 2 CAD ops, Volume: 155236 mm³



(b) Part 8 Output STEP with 4 CAD ops, Volume: 0.0070 mm³



(a) Part 6 Input with 4 CAD ops, Volume: 155236 mm³



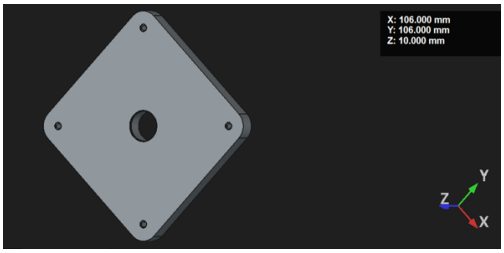
(b) Part 6 Output STEP with 8 CAD ops, Volume: 0.391 mm³

As shown in the images, the model worked well with simple objects that had

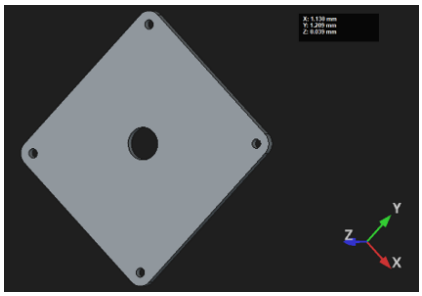
fewer CAD Operations and lower complexity. However, as the complexity and number of CAD operations increase, the model is not able to perform better. This can be seen in the last image, which is of high complexity.

4.3 Fine-tuning with geodata

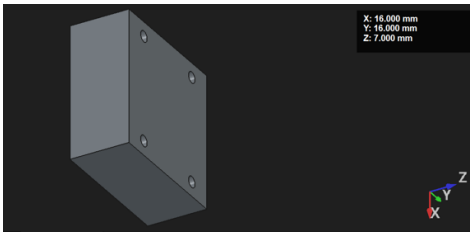
After performing the fine-tuning process as discussed in section 3.3.3, the obtained results are shown and discussed next.



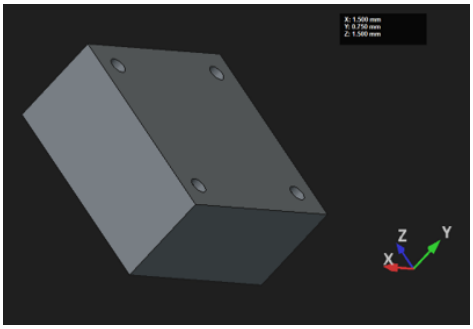
(a) Part 16 Input with 4 CAD ops, Volume: 108386.851562 mm^3



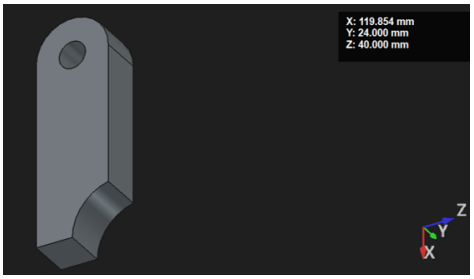
(b) Part 16 Output STEP with 3 CAD ops, Volume: : 0.051660 mm^3



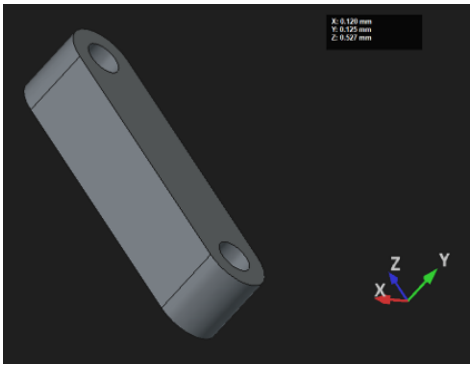
(a) Part 14 Input with 4 CAD ops, Volume: 1753.695801 mm^3



(b) Part 14 Output STEP with 2 CAD ops, Volume: 1.649905 mm^3



(a) Part 11 Input with 2 CAD ops, Volume: 98115.304688 mm^3



(b) Part 11 Output STEP with 2 CAD ops, Volume: 0.06 mm^3



(a) Part 20 Input with 7 CAD ops, Volume: 96234.101562 mm^3



(b) Part 20 Output STEP with 4 CAD ops, Volume: 0.139139 mm^3

As observed above, the model worked well with simple objects that had fewer CAD Operations and lower complexity. In fact, the model even created the object in an optimal way with less number of CAD operations than how the input was created. However, as the complexity and number of CAD operations increase, the model seems to struggle. Thus, this fine-tuned model is chosen for further analysis.

4.4 Shape optimisation results

After performing shape optimisation, the cost was reduced, and the results obtained were much better. The fine-tuning process of the model with incorporating geodata was observed, and the training loss and learning rate were calculated with each epoch and recorded. The fine-tuning was performed with low-rank Adaptation(LoRA) with the following parameters: Rank =

128, LoRA alpha = 256, LoRA dropout = 0.05, Learning rate = 1×10^{-5}, Epochs = 3. The training loss of the model is then plotted, and it can be seen below:

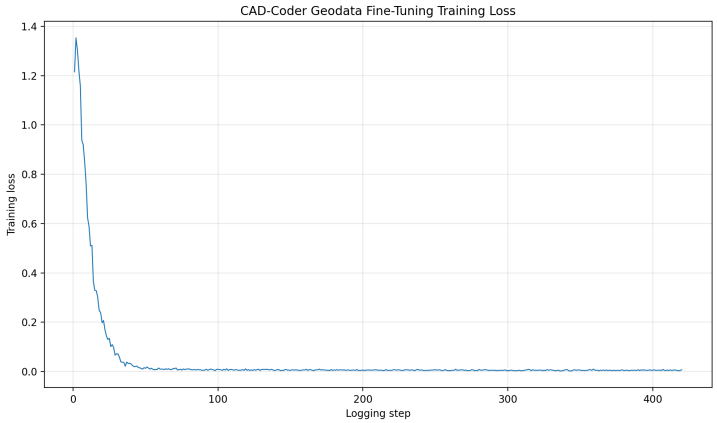


Figure 4.11: Plot of the Training Loss

As can be observed in the plot, the training loss was decreasing as the epochs increased. After this training, the output obtained was CadQuery, that was later used to perform shape optimisation, which was discussed in the previous section. After performing shape optimisation, the cost decreased from 0.085 to 0.043 approximately across 10 generations, implying that the algorithm is improving the model's geometric accuracy, as can be observed in Figure 4.12. From Figure 4.13, it can be seen that the normalised cost is significantly reduced after shape optimisation compared to before performing shape optimisation. This shows that the shape optimisation results in an improvement. The cost decreases from 0.085 to 0.043 approximately across 10 generations. The normalised cost reduces from 1.0 to approximately 0.04 for one object and to 0.06 for another object, as can be observed in the plot.

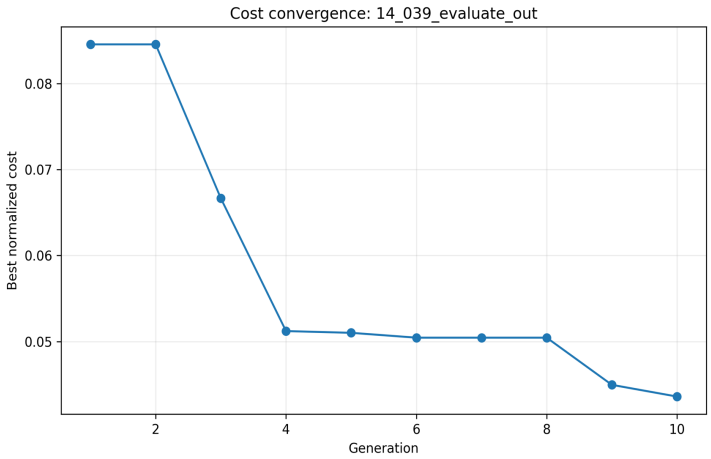


Figure 4.12: Cost Convergence Plot

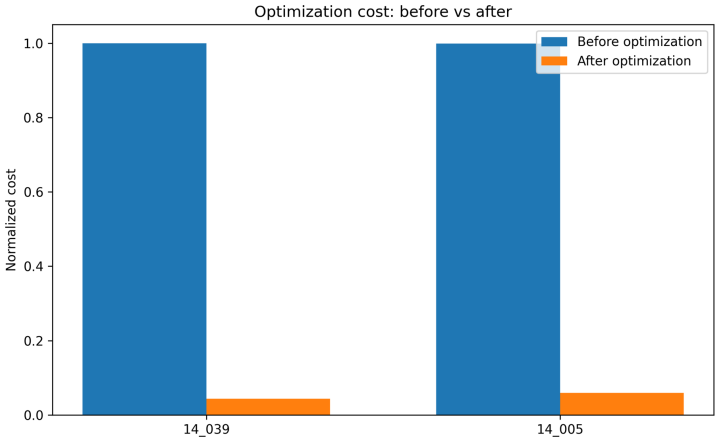


Figure 4.13: Shape optimisation Before vs After

Different values for Population and generations were used in the shape optimisation algorithm, which gave different results. In Experiment 1, the Population size is 5, and the number of Generations is 2. We started with a small population and few generations to test the optimisation pipeline while keeping the computational cost low. Later, in the second attempt, the Population size is 10, and the number of generations is 10. Both values were increased because the first experiment gave fewer candidate solutions and allowed fewer optimisation steps. The larger population resulted in more parameter combinations in each generation, whereas the more generations gave more opportunities for selection, crossover, and mutation to improve

the solutions. So, the second experiment gave better results, despite more computational effort. The comparison between 2 different population sizes and generation sizes for an object can be seen in Figure 4.14. Further analysis of the results will be explained in the next section.

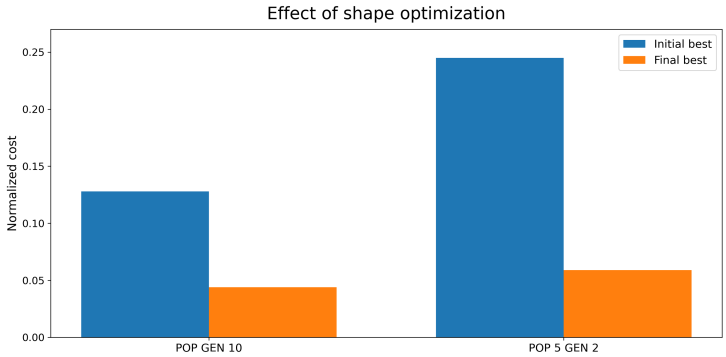


Figure 4.14: Shape Optimisation with different Populations and Generations

Object	CAD Operations	Sketch Complexity	Geometric Complexity	Number of Samples	Good Results	Percentage
Part 14	4	Simple	Simple	30	17	57%
Part 10	6	Medium	Medium	30	7	24%
Part 20	7	High	High	30	0	0%

Table 4.1: Results for objects with different CAD and geometric complexities.

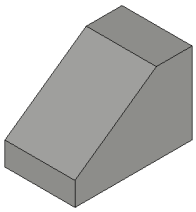
Table 4.1 shows how increasing complexity reduces the model's performance. The model performed well with simple and medium objects but struggled to reconstruct the objects as the complexity was kept increasing. Good results are those that preserve the original geometric structure and features very nicely.

Need for shape optimisation: Shape optimisation has to be performed, as there is a huge difference between the original volume and the generated volume. Before shape optimisation, from Figure3.6, the calculated Chamfer distance was 5.224730 mm, and the Hausdorff distance was 10.717521 mm. However, after performing shape optimisation, from Figure3.14, the calculated Chamfer distance was reduced to 0.243623 mm, and the Hausdorff distance was reduced to 2.350456 mm. Shape optimisation helped us obtain the most accurate results.

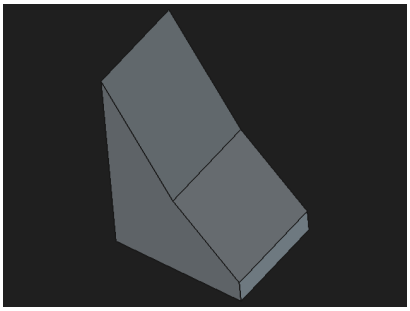
4.5 Evaluation on Unseen Data

The model was also tested on unseen data to see how well it performs. We used some of the images from the official test split of the GenCAD Code dataset, from the authors of CAD Coder, on the Hugging Face repository [8]. This ensures that the evaluation was unbiased by testing on entirely separate examples. The results are shown below:

Object 1



(a) Input object 1

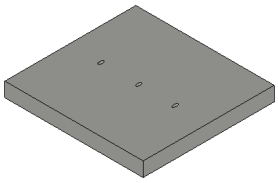


(b) Output object 1

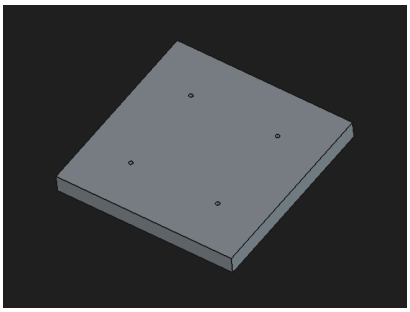
CAD Operations	Sketch Complexity	Geometric Complexity
2	Simple	Medium

Observation: The output preserves the main structure of the input, although some geometric differences remain.

Object 2



(a) Input object 2

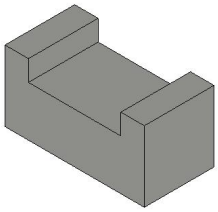


(b) Output object 2

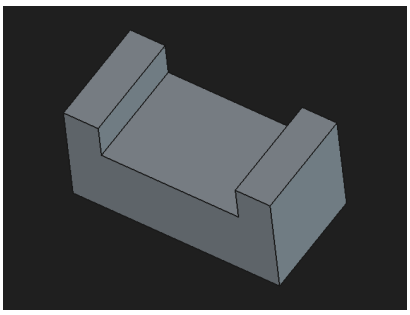
CAD Operations	Sketch Complexity	Geometric Complexity
2	Simple	Simple

Observation: The overall plate-like shape is recognisable, but the holes and proportions differ marginally.

Object 3



(a) Input object 3



(b) Output object 3

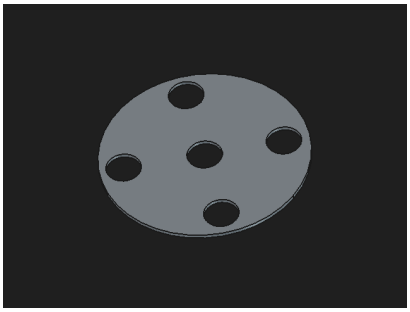
CAD Operations	Sketch Complexity	Geometric Complexity
2	Simple	Medium

Observation: The output closely matches the input, keeping the main geometric structure well preserved.

Object 4



(a) Input object 4

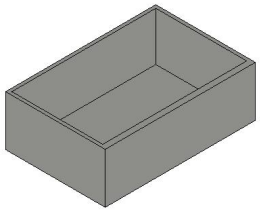


(b) Output object 4

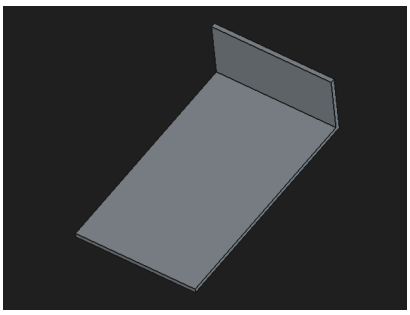
CAD Operations	Sketch Complexity	Geometric Complexity
2	High	High

Observation: The output shows apparent geometric differences, which indicate difficulties in reconstructing the complex structure.

Object 5



(a) Input object 5



(b) Output object 5

CAD Operations	Sketch Complexity	Geometric Complexity
2	High	High

Observation: The output differs from the input because it fails to properly reconstruct the hollow structure.

Chapter 5

Conclusion and Future Work

The model performed very well with simple objects, and it reconstructed them properly. In a few cases, the model reconstructed the objects in an optimal way with a small number of CAD operations. However, as the complexity increases, the model struggles to reconstruct the objects. As the sketch and geometry complexity increases, good results were not obtained. There are a few objects that have fewer CAD Operations but have high sketch and geometric complexity. And the model did not perform well, implying that the model's performance is related to the complexity nature of the object. So, the model's performance decreases with respect to an increase in complexity. The results obtained from the model must go through the shape optimisation process because of the huge error in volume. So, shape optimisation is required to reduce the volume error and obtain better results. After performing shape optimisation, we get better results, and analysis can be performed. To conclude, it can be said that the ability of the model to reconstruct a parametric CAD code from the given STL object depends on the complexity of the object, and better results can be obtained through shape optimisation.

The future scope of this project would be to improve the performance with respect to complex objects, as complex geometries are difficult to reconstruct accurately. The generated CAD code requires optimisation. So, work can be done to automatically extract CAD parameters and improve the optimisation process.

References

[1] Aminu Abdullahi and Kathryn Timonera: *Large Language Model: A Guide to the Question "What Is an LLM?"* https://www.eweek.com/artificial-intelligence/large-language-model/. Accessed: 2025-10-16. (2024).

[2] Md Ferdous Alam and Faez Ahmed: GenCAD: Image-Conditioned Computer-Aided Design Generation with Transformer-Based Contrastive Representation and Diffusion Priors. In: *arXiv preprint arXiv:2409.16294* (2024). DOI: 10.48550/arXiv.2409.16294. URL: https://arxiv.org/abs/2409.16294.

[3] Shuai Bai et al.: Qwen2.5-VL Technical Report. In: *arXiv preprint arXiv:2502.13923* (2025). DOI: 10.48550/arXiv.2502.13923. URL: https://arxiv.org/abs/2502.13923.

[4] H. M. Cartwright: An Introduction to Evolutionary Computation and Evolutionary Algorithms. In: *Applications of Evolutionary Computation in Chemistry*. Ed. by R. L. Johnston. Vol. 110. Structure and Bonding. Berlin and Heidelberg: Springer, (2004), pp. 3–36. DOI: 10.1007/b13931. URL: https://doi.org/10.1007/b13931.

[5] Anna C. Doris et al.: CAD-Coder: An Open-Source Vision-Language Model for Computer-Aided Design Code Generation. In: *arXiv preprint arXiv:2505.14646* (2025). DOI: 10.48550/arXiv.2505.14646. URL: https://arxiv.org/abs/2505.14646.

[6] Edward J. Hu et al.: LoRA: Low-Rank Adaptation of Large Language Models. In: *arXiv preprint arXiv:2106.09685* (2021). DOI: 10.48550/arXiv.2106.09685. URL: https://arxiv.org/abs/2106.09685.

[7] Mohammad Sadil Khan et al.: Text2CAD: Generating Sequential CAD Models from Beginner-to-Expert Level Text Prompts. In: *arXiv preprint arXiv:2409.17106* (2024). Accepted at NeurIPS 2024 (Spotlight). DOI: 10.48550/arXiv.2409.17106. URL: https://arxiv.org/abs/2409.17106.

[8] *Mesh-To-CAD AI: Last Year Report and Code*. Project report and code.

[9] Carl D. Meyer: *Matrix Analysis and Applied Linear Algebra*. SIAM, (2001). URL: http://www.matrixanalysis.com/.

[10] Waleed Mohamed et al.: 3D Reconstruction from 2D Multi-View Dental 2D Images Based on EfficientNetB0 Model. In: *Scientific Reports* 15.1 (2025). DOI: 10.1038/s41598-025-12861-3. URL: https://doi.org/10.1038/s41598-025-12861-3.

[11] Felix Ocker et al.: From Idea to CAD: A Language Model-Driven Multi-Agent System for Collaborative Design. In: (2025). arXiv preprint arXiv: 2503.04417. DOI: 10.48550/arXiv.2503.04417. URL: https://arxiv.org/abs/2503.04417.

[12] OpenAI: *ChatGPT: Python Code for CAD Sequence*. https://chatgpt.com/share/68f9e984-5740-800b-b3f7-0edb591b547d. Accessed: 2025-10-23. (2025).

[13] Rainer Storn and Kenneth Price: Differential Evolution—A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces. In: *Journal of Global Optimization* 11.4 (1997), pp. 341–359. DOI: 10.1023/A:1008202821328. URL: https://doi.org/10.1023/A:1008202821328.

[14] M. Szilvsi-Nagy and Gyula Mátyási: Analysis of STL Files. In: *Mathematical and Computer Modelling* (2003). DOI: 10.1016/S0895-7177(03)90079-3. URL: https://doi.org/10.1016/S0895-7177(03)90079-3.

[15] Yang Zhao, Hao Zhang, and Xiuyuan Hu: Role Taxonomy of Units in Deep Neural Networks. In: *Proceedings of UniReps: The First Workshop on Unifying Representations in Neural Models*. Vol. 243. Proceedings of Machine Learning Research. PMLR, (2024), pp. 291–301. URL: https://proceedings.mlr.press/v243/zhao24b.html.